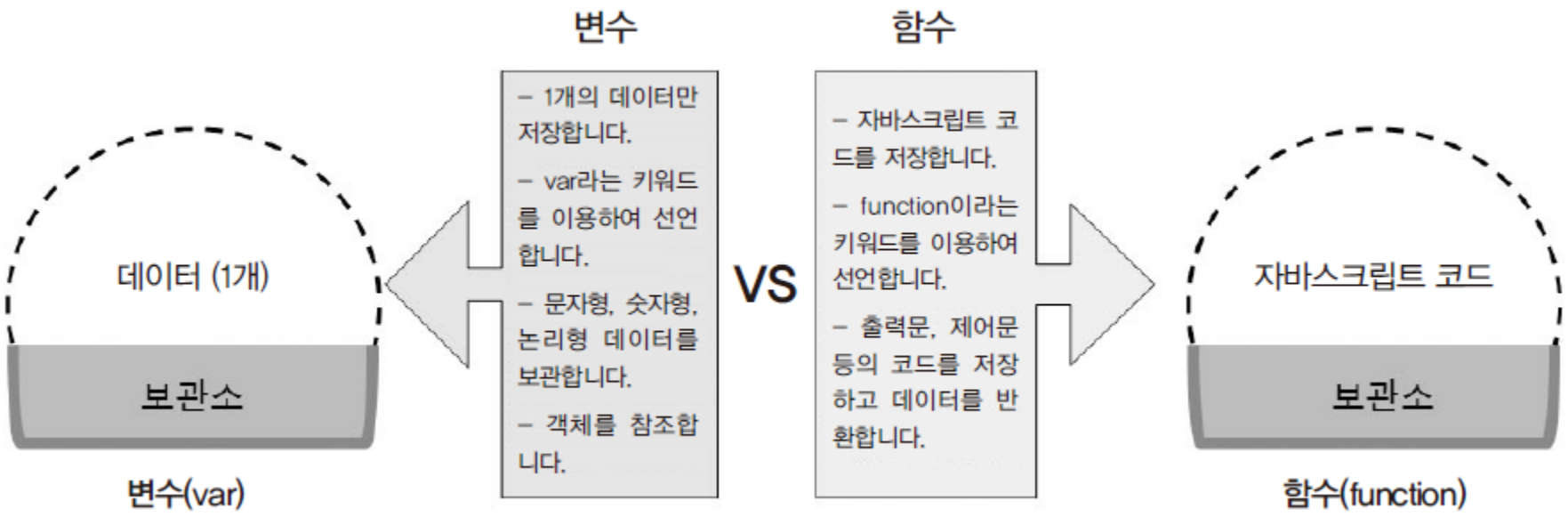
5장 함수

05-1 함수

함수란?


변수
함수
데이터 (1개)
보관소
변수(var)
– 1개의 데이터만 저장합니다.
– var라는 키워드를 이용하여 선언합니다.
– 문자형, 숫자형, 논리형 데이터를 보관합니다.
– 객체를 참조합니다.
VS
– 자바스크립트 코드를 저장합니다.
– function이라는 키워드를 이용하여 선언합니다.
– 출력문, 제어문 등의 코드를 저장하고 데이터를 반환합니다.
자바스크립트 코드
보관소
함수(function)

기본 함수 정의문 — 정의, 사용

```
function myFnc(  ) {
    document.write( "hello~", "<br>" );
    document.write( "welcome",  "<br>" );
}

myFnc(  );

myFnc(  );
```

함수가 2회 호출되어 코드 내용을 2회 실행합니다.

기본 함수 정의문 — 정의, 사용

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset = "UTF-8">
<title> 함수 </title>
<script>
var count = 0;

myFnc();
function myFnc() {
count++;
document.write("hello" + count,"<br>");
}

myFnc();

var theFnc = function() {
count++;
document.write("bye" + count,"<br>");
}

theFnc();
</script>
</head>
<body>
</body>
</html>
```

명명함수는 선언 전에 호출하더라도 가능함

예약어

컴파일 기준으로 생성

실행 기준으로 생성

기본 함수 정의문 — 정의, 사용

```
<html>
<head>
<script language="javascript">
<!--
var a = 5, b, c;
function aaa() {
b = a + 5;
document.write("b = " + b + "<br>");
}
function bbb() {
c = b + 5;
document.write("c = " + c + "<br/>");
}
//-->
</script>
</head>
<body>
<script>
aaa();
bbb();
</script>
</body>
</html>
```

선언부와 호출부를 분리해서 오류가 발생하지 않도록 한다.

기본 함수 정의문 — 정의, 사용

```
<html>
<head>
<script language="javascript">
<!--
function aaa() {
var a = 5;
var b = a + 5;
var c = b + 5;
document.write("b = " + b + "<br/>");
document.write("c = " + c + "<br/>");
}
//-->
</script>
</head>
<body>
<script>
aaa();
</script>
</body>
</html>
```

a,b,c전역변수를 블록 안 지역변수화 해줌

하나의 기능 -> 연관된 기능을 묶어줌

일반 함수 정의 vs 익명 함수 선언 참조

전문가의 조언

일반 함수 정의 방식과 익명 함수 선언 참조 방식의 차이점

일반 함수 정의는 함수 호출 시 호이스팅(hoisting) 기술을 지원합니다. 그러나 익명 함수 선언 참조 방식은 호이스팅을 지원하지 않습니다. 호이스팅을 적용하면 함수 정의문보다 호출문이 먼저 나와도 함수 정의문을 끌어올려 함수를 호출합니다.

A. 일반 함수 정의 방식 (정상 작동)	B. 익명 함수 선언 참조 방식 (오류 발생)

ⓐ testFnc( ); ← 호출문
 	
호이스팅(hoisting)
 	
ⓑ function testFnc( ) {
ⓒ 자바스크립트 코드;
} ← 함수 정의문
	ⓓ testFnc( );
	
	ⓔ var testFnc = function ( ) {
	ⓕ 자바스크립트 코드;
	}

매개변수가 있는 함수 정의문

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset = "UTF-8">
<title> 함수 </title>
<script>
var rightId = "korea";
var rightPw = "1234";
function login(id, pw){
if(id == rightId){
if(pw == rightPw){
document.write(id + "님 방문을 환영합니다");
}else{
alert("잘못된 비밀번호입니다.");
}

} else {
alert("존재하지 않는 아이디입니다.");
}
}

var userId = prompt("아이디를 입력하세요.", "");
var userPw = prompt("패스워드를 입력하세요.", "");

login(userId, userPw);
</script>
</head>
<body>
</body>
</html>
```

매개변수? 인자? 파라미터

function 함수명(①매개변수 1, ②매개변수 2,…③매개변수 n)
  자바스크립트 코드;
}
함수명(ⓐ데이터 1, ⓑ데이터 2,…ⓒ데이터 n);

매개변수가 있는 함수 정의문 활용

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset = "UTF-8">
<title> 함수 </title>
<script>
var rightId = "ehtm"
varrightPw = "1234"
function login(){
if(fom.id.value == rightId){
if(fom.pwd.value == rightPw){
document.write(fom.id.value + "님 방문을 환영합니다");
}else{
alert("잘못된 비밀번호입니다.");
}

} else {
alert("존재하지 않는 아이디입니다.");
location.reload();
}
}
</script>
</head>
```

접근 연산자 name을 통해 input으로 받은 value를 가져옴

```
<body>
<form name = "fom" >
<h1  style="text-align:center">회원 가입창</h1>
<table  align="center">
    <tr>
       <td width="200"><p align="right">아이디</td>
       <td width="400"><input type="text" name="id"></td>
    </tr>
    <tr>
        <td width="200"><p align="right">비밀번호</td>
        <td width="400"><input type="password"  name="pwd"></td>
    </tr>
    <tr>
        <td width="200"><p> </p></td>
        <td width="400">
       <input type="submit" value="로그인하기" onclick="login()">
       </td>
    </tr>
</body>
</html>
```

05-2 함수에서 return 문의 역할

데이터를 반환하고 강제 종료하는 return 문

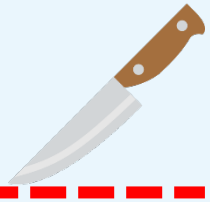
기본형

```
function 함수명( ) {

    자바스크립트 코드1;   ❷
    return 데이터(값);    ❸

    자바스크립트 코드2;
}

var 변수 = 함수명( );     ❶
```



❶ 함수 호출문

❷ 함수 코드 실행

❸ 자바스크립트 코드2를 무시하고 데이터를 반환

데이터를 반환하고 강제 종료하는 return 문

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset = "UTF-8">
<title> 함수 </title>
<style>
  *{margin: 0;padding: 0;}
  #galleryZone {text-align: center;}
  #galleryZone input[type="image"]{margin:0 8px;}
</style>
<script>
  var num = 1;
  function gallery(direct) {
    if(direct) {
      if(num == 8) return;
      num++;
    } else {
      if(num == 1) return;
      num--;
    }

    var imgTag = document.getElementById("photo");
    imgTag.setAttribute("src","images/pic_" + num + ".jpg");
  }
</script>
</head>
<body>
  <div id="galleryZone">
    <p><img src="images/pic_1.jpg" id="photo"
alt=""></p>
    <p>
      <button onclick="gallery(0)">이전</button>
      <button onclick="gallery(1)">다음</button>
    </p>
  </div>
</body>
</html>
```

전역변수

해당 id값을 가져와라

속성을 만들어라

return 문 활용

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset = "UTF-8">
<title> 함수 </title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.min.css" rel="stylesheet"
integrity="sha384-rbsA2VBKQhggwzxH7pPCaAqO46MgnOM80zW1RWuH61DGLwZJEdK2Kadq2F9CUG65" crossorigin="anonymous">
<style>
*{margin: 0;padding: 0;}
#galleryZone {text-align: center;}
#galleryZone input[type="image"]{margin:0 8px;}

</style>
<script>
var num = 1;
function gallery(direct) {
if(direct) {
if(num == 8) return;
num++;
} else {
if(num == 1) return;
num--;
}

var imgTag = document.getElementById("photo");
imgTag.setAttribute("src","images/math_img_" + num + ".jpg");
}
</script>
</head>
<body>
<!-- 부트스트랩 -->
<div id="galleryZone" >
<p><img src="images/math_img_1.jpg" id="photo" alt=""
class="border border-warning border-5"></p>
<p>
<button type="button" class="btn btn-success btn-lg"
onclick="gallery(0)">이전</button>
        <button type="button" class="btn btn-danger
btn-lg" onclick="gallery(1)">다음</button>
</p>
</div>
</body>
</html>
```

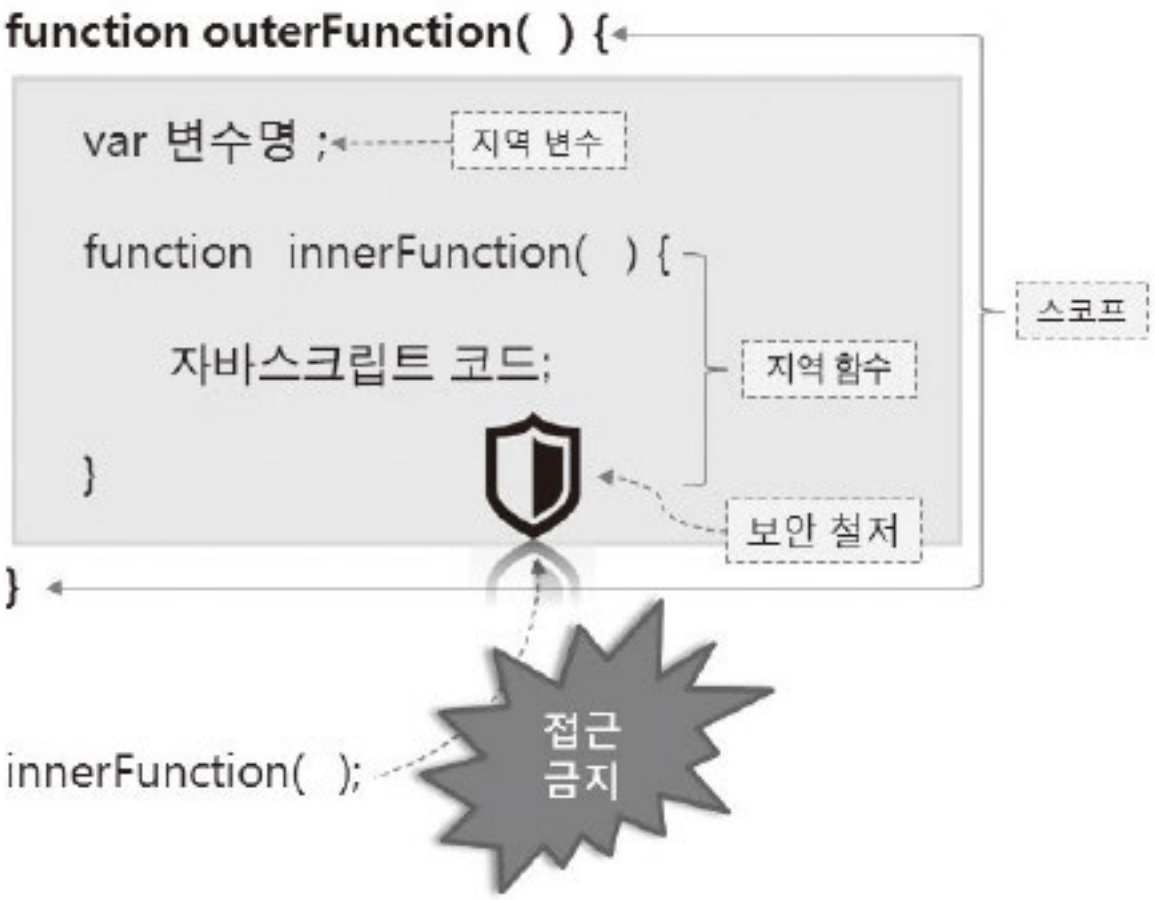
재귀 함수 호출

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset = "UTF-8">
<title> 함수 </title>
<script>
  var num = 0;
  function testFnc(){
    num++;
    document.write(num, "<br>");
    if(num == 10) return;
    testFnc();
  }
  testFnc();
</script>
</head>
<body>
</body>
</html>
```

```
function myFnc(){
  자바스크립트 코드;
  myFnc();
}
myFnc();
```

05-3 함수 스코프 개념 이해

함수 스코프란?


function outerFunction( ) {
var 변수명 ;
지역 변수
function innerFunction( ) {
자바스크립트 코드;
}
지역 함수
스코프
보안 철저
}
innerFunction( );
접근
금지

전역 변수와 지역 변수의 개념과 차이

기본형

```
var 변수명;        // 전역 변수

function 함수명( ) {
  var 변수명;      // 지역 변수
}
```

전역 함수와 지역 함수의 차이

기본형

```
function 함수명1( ) {
   자바스크립트 코드;
}                          // 전역 함수

function 함수명2( ) {
   function 함수명3( ) {
      자바스크립트 코드;
   }                       // 지역 함수
}
```

05-4 객체 생성자 함수의 활용

객체 생성자 함수

기본형

```
function 함수명(매개변수1, 매개변수2, ...매개변수n) {      //객체 생성자 함수
    this.속성명 = 새 값;
    this.함수명 = function( ) {
        자바스크립트 코드;
    }
}

var 참조 변수(인스턴스 네임) = new 함수명( );                //객체 생성
                              ↓
var 참조 변수 = { 속성 : 새 값, 함수명 : function( ) {  ...  } }
```

객체 생성자 함수

```
<html>
<head>
<script language="javascript">
<!--
function ranData() {
var date = new Date();
var x = date.getSeconds();
var y = (x % 5) + 1;
alert("난수 = " + y);
}
//-->
</script>
</head>
<body>
<a href="#" onClick="ranData()"> 난수 발생 시키기 </a>
</body>
</html>
```

05-5 자바스크립트 내장 함수

내장 함수

종류	설명	사용 예
encodeURI( )	문자를 유니 코드값으로 인코딩합니다. (영문, 숫자, 일부 기호(; , / ? : @ & = + $)는 제외)	encodeURI("?query=값"); → "?query=%EA%B0%91"
encodeURIComponent( )	문자를 유니 코드값으로 인코딩합니다(영문, 숫자 제외).	encodeURIComponent("?query=값") → "%3Fquery%3D%EA%B0%91"
decodeURI( )	유니 코드값을 디코딩해 다시 문자화합니다.	decodeURI("?query=%EA%B0%91") → "?query=값"
decodeURIComponent( )	유니 코드값을 디코딩해 다시 문자화합니다.	decodeURIComponent("%3Fquery%3D%EA%B0%91") → "?query=값"
parseInt( )	문자열 데이터를 정수형 데이터로 반환합니다.	parseInt("5.12") → 5 parseInt("15px") → 15
parseFloat( )	문자열 데이터를 실수형 데이터로 반환합니다.	parseInt("5.12") → 5.12 parseInt("65.5%") → 65.5
String( )	문자형 데이터로 반환합니다.	String(5) → "5"
Number( )	숫자형 데이터로 반환합니다.	Number("5") → 5
Boolean( )	논리형 데이터로 반환합니다.	Boolean(5) → true Boolean(null) → false
isNaN( )	is Not a Number의 약자이며 숫자가 아닌 문자가 포함되어 있으면 true를 반환합니다.	isNaN("5–3") → true isNaN("53") → false
eval( )	문자형 데이터를 따옴표가 없는 자바스크립트 코드로 처리합니다	eval("15 + 5") → 20

내장 함수

```
var number = '100';
alert(parseInt(number, 2)); // 4
alert(parseInt(number, 8)); // 64
alert(parseInt(number, 10)); // 100
alert(parseInt(number, 16)); // 256


var number = '52.273e5';
alert(parseFloat(number)); // 5227300


var number = "일";

if (isNaN(number) == true) {
alert("숫자가 아닙니다");
} else {
alert("숫자 입니다");
}
```